

Aula 03

Persistência com Spring Data JPA

Objetivos

Entender o que é o JPA e por que ele existe

Transformar a classe Usuario em uma entidade persistente

Criar um Repository e substituir a lista em memória por um banco de dados real (H2)

Agenda

1. Revisão
2. Conceitos
3. Demonstração
4. Prática
5. Exercícios

Revisão da Aula Anterior

Criamos uma API REST com Spring Boot (`@RestController` , `@GetMapping`)

Modelamos a classe Usuario (nome, username, senha, email)

Usamos `@JsonIgnore` para ocultar a senha na resposta JSON

Os dados de Usuario estavam **em memória** (`ArrayList`), perdidos a cada reinício da aplicação

O Problema de Hoje

Na Aula 02, o método `getAllUsuarios()` criava a lista de usuários **toda vez** que o endpoint era chamado:

```
List<Usuario> users = new ArrayList<>();  
users.add(new Usuario("João","joao123","senha123","joao@example.com"));
```

Não há **persistência**: os dados não sobrevivem a um reinício da aplicação

Não é possível **inserir, atualizar ou excluir** um usuário de verdade

Precisamos de um banco de dados

Conceitos Fundamentais

O que é JPA?

Java Persistence API: especificação Java para persistência de dados em bancos relacionais

Define uma **interface** e um conjunto de **regras** para mapear objetos Java (classes) para tabelas de banco de dados

Não é uma implementação, é um padrão — quem implementa é o **Hibernate** (usado por padrão pelo Spring Boot)

Permite trabalhar com **objetos** em vez de escrever SQL manualmente (ORM — *Object-Relational Mapping*)

Principais Conceitos do JPA

Entity: classe Java que representa uma tabela no banco de dados. Anotada com `@Entity`

EntityManager: interface para interagir com o contexto de persistência (salvar, buscar, atualizar, remover)

JPQL: linguagem de consulta orientada a objetos, parecida com SQL, mas trabalha com entidades

Repository: no Spring Data JPA, uma interface que abstrai o acesso a dados, eliminando a necessidade de escrever a maior parte do SQL/JPQL manualmente

Principais Anotações

@Entity

@Id

@GeneratedValue

@Basic

@Column

@Transient

@Entity : marca a classe como uma tabela

@Id : marca o atributo como chave primária

@GeneratedValue : define a estratégia de geração automática do @Id

@Column : configurações adicionais da coluna (nome, tamanho, obrigatoriedade, etc.)

Spring Boot + React 7

Programação Web II

Restrições das Entidades

Para o Hibernate conseguir mapear a classe corretamente, ela precisa: Ter um **construtor padrão** (sem argumentos)

Ter **getters e setters** para os atributos mapeados

Seguir o padrão **CamelCase**

Ser um **POJO** (*Plain Old Java Object* — sem lógica de negócio complexa) Spring Boot +

React 8

Programação Web II

Arquitetura / Fluxo

Requisição HTTP



Controller (@RestController)



Repository (JpaRepository<Usuario, Long>)



EntityManager / Hibernate



Banco de Dados (H2 / PostgreSQL)

O Spring Data JPA gera a implementação do Repository automaticamente em tempo de execução.

Spring Boot + React 9

Programação Web II

Transformando **Usuario** em Entidade

Relembrando a classe criada na Aula 02:

```
public class Usuario {
```

```
private String nome;  
private String username;  
private String senha;  
private String email;
```

```
// Getters e Setters
```

```
}
```

Agora vamos adicionar as anotações do JPA.

Usuario.java (com JPA)

```
@Entity  
public class Usuario {  
  
    @Id  
    @GeneratedValue(strategy = GenerationType.IDENTITY)  
    private Long id;
```

```
private String nome;  
private String username;  
  
@JsonIgnore  
private String senha;  
  
private String email;  
  
public Usuario() {  
}  
  
public Usuario(String nome, String username, String senha, String email) {  
    this.nome = nome;  
    this.username = username;  
    this.senha = senha;  
    this.email = email;  
}
```

// Getters e Setters

}

Programação Web II

Spring Boot + React 11

Dependências no pom.xml

```
<dependencies>
```

```
<!-- ... -->
```

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
  <groupId>com.h2database</groupId>
  <artifactId>h2</artifactId>
</dependency>
</dependencies>
```

Se o projeto foi criado no Aula 01 já com "Spring Data JPA" e "H2 Database", essas dependências já estão presentes.

Spring Boot + React 12

Programação Web II

application.properties

```
spring.datasource.url=jdbc:h2:file:./database.db  
spring.datasource.driverClassName=org.h2.Driver  
spring.datasource.username=sa  
spring.datasource.password=
```

```
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect  
spring.jpa.hibernate.ddl-auto=update
```

ddl-auto=update : o Hibernate cria/atualiza as tabelas automaticamente com base nas entidades

O H2 é um banco em arquivo, ótimo para desenvolvimento e testes Spring Boot +

React 13

Criando o Repository

```
public interface UsuarioRepository extends JpaRepository<Usuario, Long> { }
```

JpaRepository<Usuario, Long> : Usuario é a entidade, Long é o tipo do @Id

Já fornece pronto: save() , findAll() , findById() , deleteById() , entre outros

Nenhuma implementação precisa ser escrita — o Spring Data JPA gera tudo em tempo de execução

Demonstração

UserController.java — antes (Aula 02)

```
@RestController
@RequestMapping("/api")
public class UserController {

    @GetMapping("/usuarios")
    public List<Usuario> getAllUsuarios() {
        List<Usuario> users = new ArrayList<>();
        users.add(new Usuario("João","joao123","senha123","joao@example.com")); users.add(new
        Usuario("Maria","maria456","senha456","maria@example.com")); return users;
    }
}
```

UsuarioController.java — depois (com JPA)

```
@RestController
@RequestMapping("/api")
public class UsuarioController {

    @Autowired
    private UsuarioRepository usuarioRepository;

    @GetMapping("/usuarios")
    public List<Usuario> getAllUsuarios() {
        return usuarioRepository.findAll();
    }

    @PostMapping("/usuarios")
    public Usuario createUsuario(@RequestBody Usuario usuario) {
        return usuarioRepository.save(usuario);
    }
}
```

Buscando um Usuário por Id

```
@GetMapping("/usuarios/{id}")  
public Usuario getUsuarioById(@PathVariable Long id) {  
    return usuarioRepository.findById(id)  
        .orElseThrow(() -> new RuntimeException("Usuário não encontrado")); }
```

findById() retorna um `Optional<Usuario>`

orElseThrow() lança uma exceção caso o id não exista (trataremos exceções melhor na Aula 09)

Excluindo um Usuário

```
@DeleteMapping("/usuarios/{id}")  
public void deleteUsuario(@PathVariable Long id) {  
    usuarioRepository.deleteById(id);  
}
```

deleteById() remove o registro do banco pelo @Id

Vamos aprofundar o CRUD completo (POST, PUT, DELETE) na Aula 05 Spring Boot +

Testando com o H2 Console (opcional)

```
spring.h2.console.enabled=true  
spring.h2.console.path=/h2-console
```

Acessar <http://localhost:8080/h2-console> e usar a mesma URL configurada em `spring.datasource.url` para visualizar as tabelas e os dados diretamente no navegador.

Atividade Prática

Objetivo

Persistir a classe Usuario criada na Aula 02 em um banco H2 real, substituindo a lista em memória.

Passos

1. Adicionar as anotações `@Entity` , `@Id` e `@GeneratedValue` na classe Usuario 2.

Configurar o `application.properties` com o H2

3. Criar a interface `UsuarioRepository` extends `JpaRepository<Usuario, Long>` 4. Alterar o `UsuarioController` para usar o `UsuarioRepository` no lugar do `ArrayList`

5. Testar os endpoints `GET /api/usuarios` e `POST /api/usuarios` (via `Spring Boot + React 20`

Programação Web II

Exercícios

1. Repetir o processo para a classe `Permissao` criada no exercício da Aula 02 (`id` , `nome` , `descricao`): transformá-la em `@Entity` e criar `PermissaoRepository`
2. Criar os endpoints `POST /permissoes` (inserir) e `GET /permissoes/{id}` (buscar por id)
3. Repetir o mesmo processo para a classe/entidade de seu interesse escolhida no exercício da Aula 02 (`Produto` , `Pedido` , `Cliente` , etc.)

4. Ao final, comitar e subir as alterações no repositório do GitHub

Spring Boot + React 21

Programação Web II

Referências

Spring Boot Documentation

Spring Data JPA Documentation

React Documentation

Material complementar da disciplina

<https://docs.spring.io/spring-data/jpa/reference/>

<https://www.baeldung.com/the-persistence-layer-with-spring-data-jpa> Spring Boot +

React 22

Programação Web II

Próxima Aula

Conteúdo: Fundamentos do React (componentes, props, state, hooks) para começar a consumir a API de Usuario no front-end

Preparação: deixar o backend rodando com o CRUD de Usuario funcionando via H2

Spring Boot + React 23